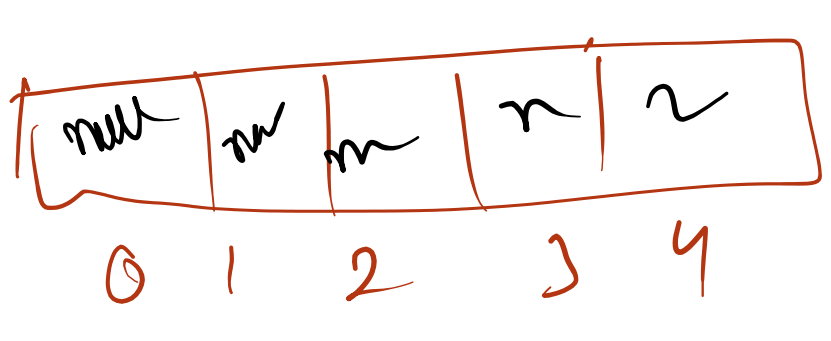
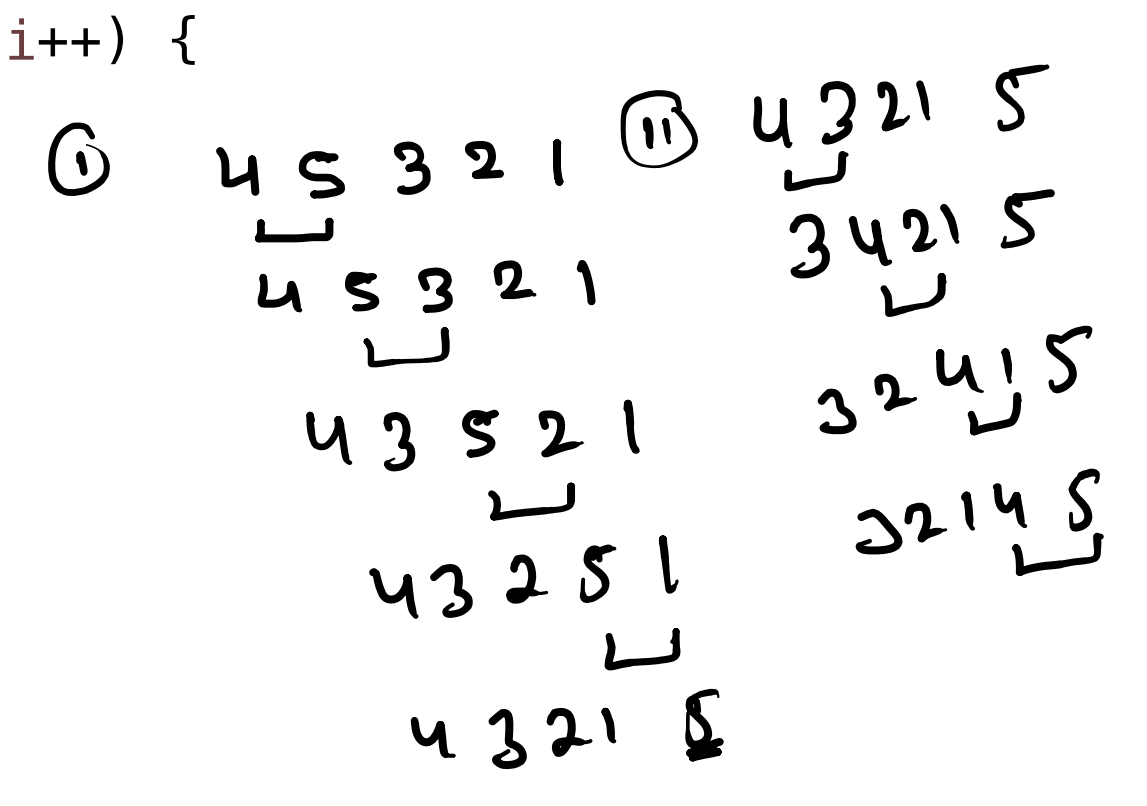
Lec-29

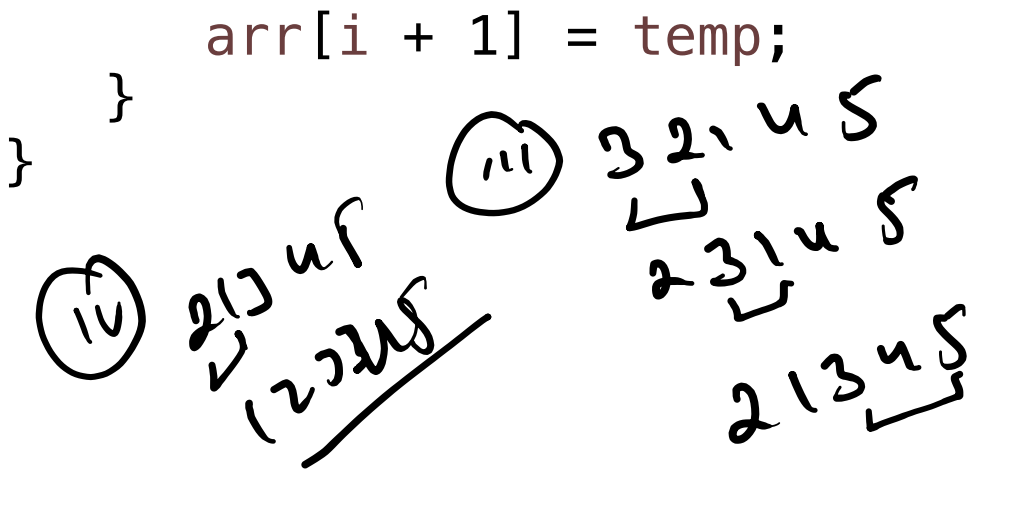
31/08/25 9:17 AM

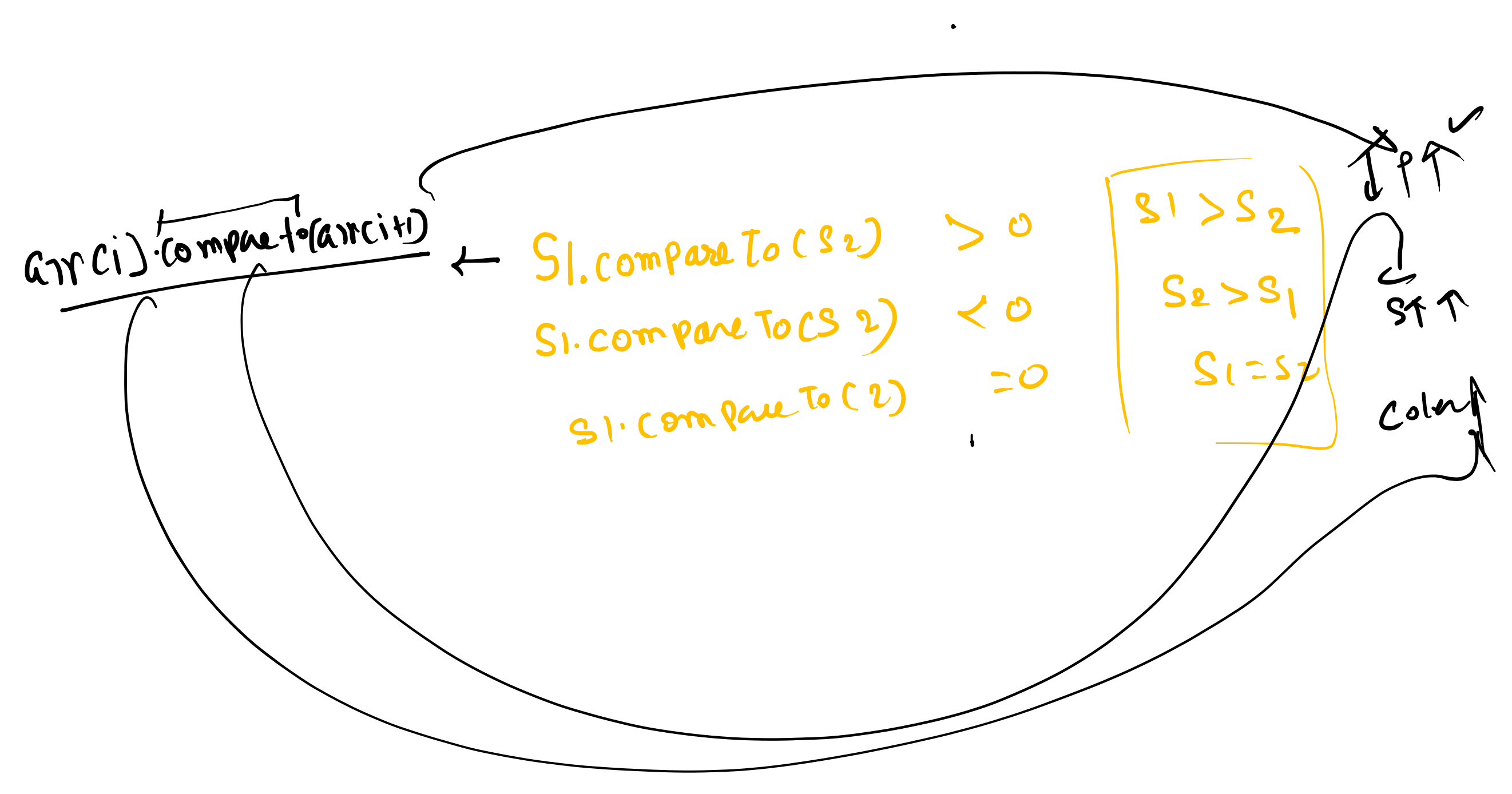
```
Cars[] arr = new Cars[5];
```


null
n
n
n
n
0
1
2
3
4

```
public static void Sort(int[] arr) {
        // TODO Auto-generated method stub
        for (int turn = 1; turn < arr.length; turn++) {
            for (int i = 0; i <arr.length-turn ;i++) {
                if (arr[i] > arr[i + 1]) {
                    int temp = arr[i];
                    arr[i] = arr[i + 1];
                    arr[i + 1] = temp;
                }
            }
        }
    }
```


(i) 4 5 3 2 1
4 5 3 2 1
4 3 5 2 1
4 3 2 5 1
4 3 2 1 5
(ii) 4 3 2 1 5
3 4 2 1 5
3 2 4 1 5
3 2 1 4 5

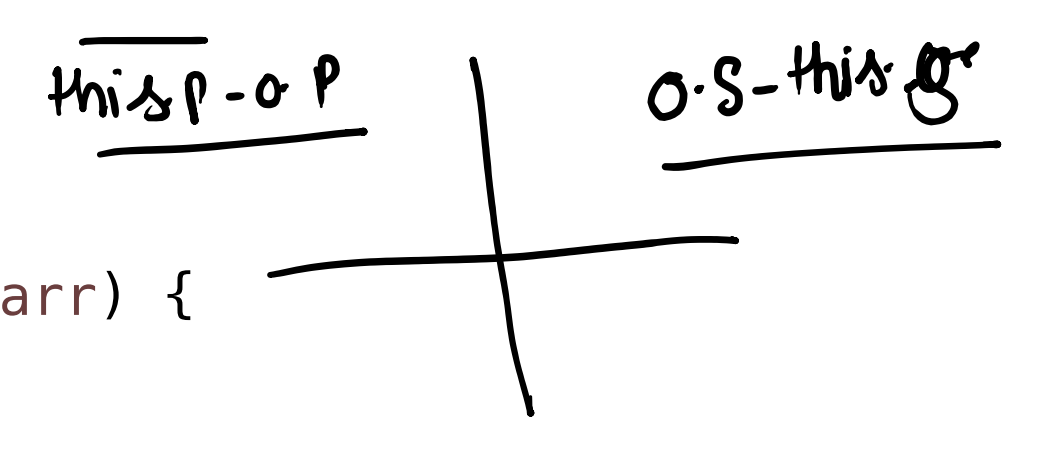

(iii) 3 2 1 4 5
2 3 1 4 5
2 1 3 4 5
(iv) 2 1 3 4 5
1 2 3 4 5


arr[i].compareTo(arr[i+1])
S1.compareTo(S2) > 0
S1.compareTo(S2) < 0
S1.compareTo(2) = 0
S1 > S2
S2 > S1
S1 = S2
P↑
St↑
Color

this→ [345, 3, Yellow] o→ [34, 89, Grey]

```
@Override
    public int compareTo(Cars o) {
        // TODO Auto-generated method stub
        return 0;
    }
```

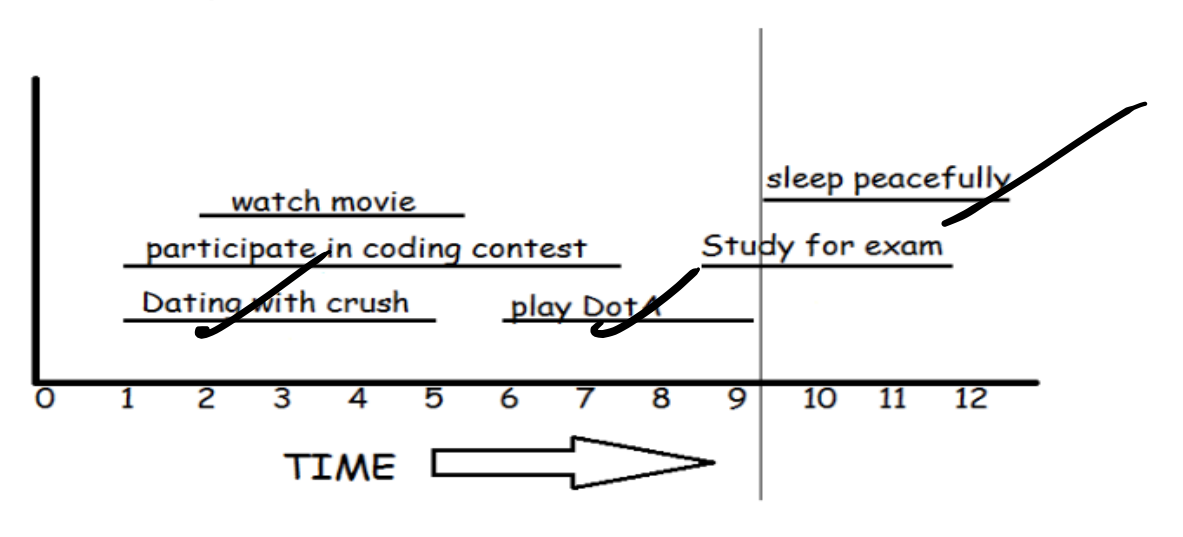
S1.compare(S2)

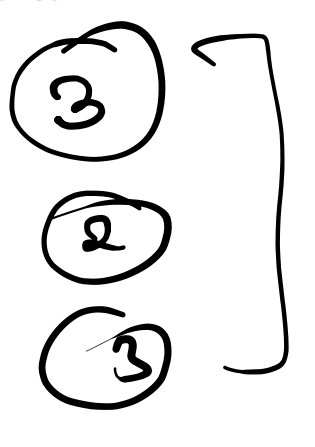

this.P - o.P
o.S - this.S

```
public static <T extends Comparable<T>> void Sort(T[] arr) {
        // TODO Auto-generated method stub
        for (int turn = 1; turn < arr.length; turn++) {
            for (int i = 0; i < arr.length - turn; i++) {
                if (arr[i].compareTo(arr[i + 1])>0) {
                    T temp = arr[i];
                    arr[i] = arr[i + 1];
                    arr[i + 1] = temp;
                }
            }
        }
    }
```

```
Cars[] arr = new Cars[5];
arr[0] = new Cars(200, 10, "White");// P S C
arr[1] = new Cars(1000, 20, "Black");
arr[2] = new Cars(345, 3, "Yellow");
arr[3] = new Cars(34, 89, "Grey"); SK
arr[4] = new Cars(8907, 6, "Red");
```

You are actually very busy man. You have a big schedule of activities. Your aim is to do as much as activities as possible.





In the given figure, if you go to date with crush, you cannot participate in the coding contest and you can't watch the movie. Also if you play DotA, you can't study for the exam. If you study for the exam you can't sleep peacefully. The maximum number of activities that you can do for this schedule is 3.

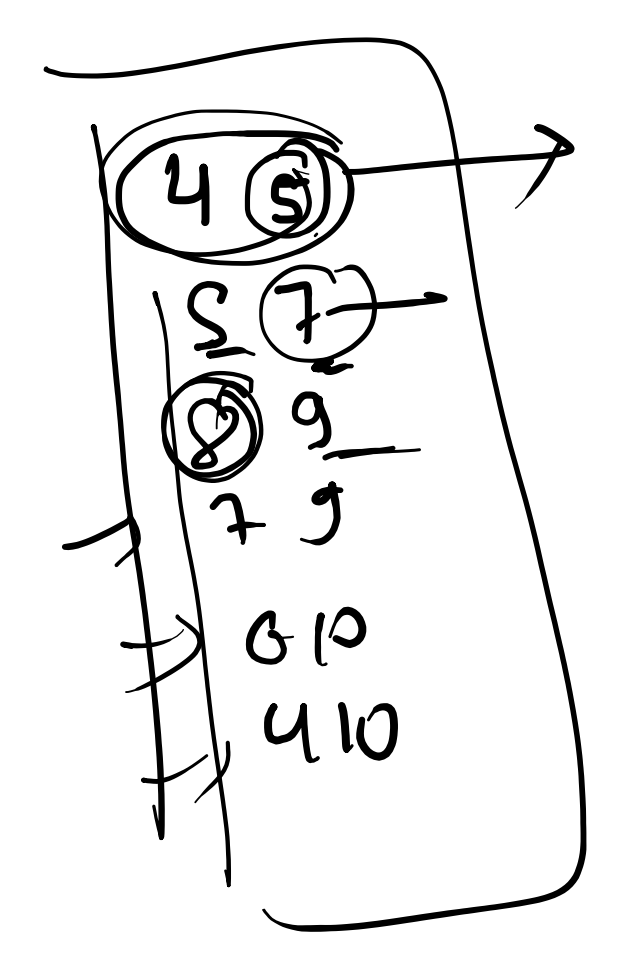
Either you can

- watch movie, play DotA and sleep peacefully (or)
- date with crush, play DotA and sleep peacefully

Input

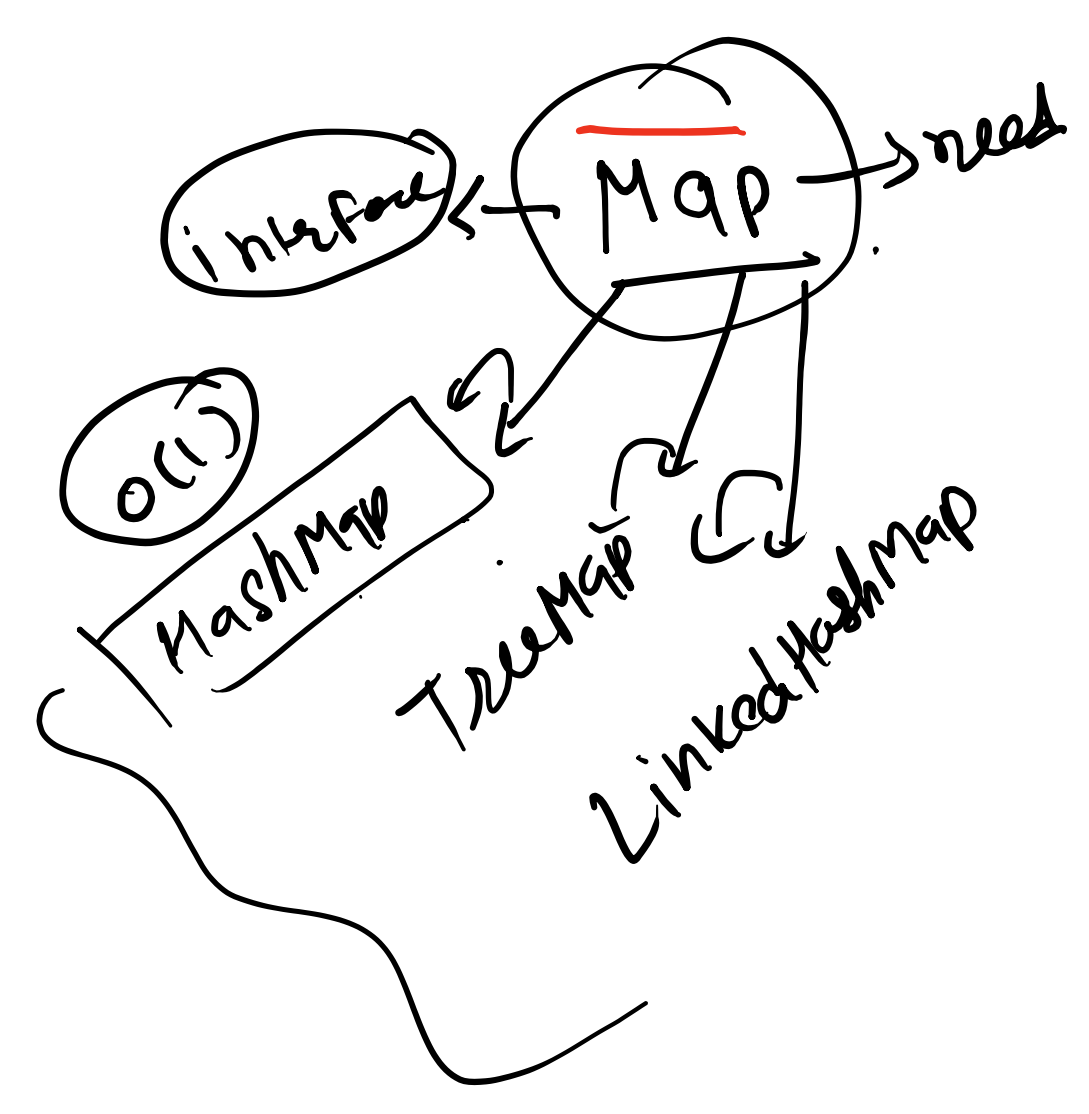
The first line consists of an integer **T**, the number of test cases. For each test case the first line consists of an integer **N**, the number of activities. Then the next **N** lines contains two integers **m** and **n**, the start and end time of each activity.

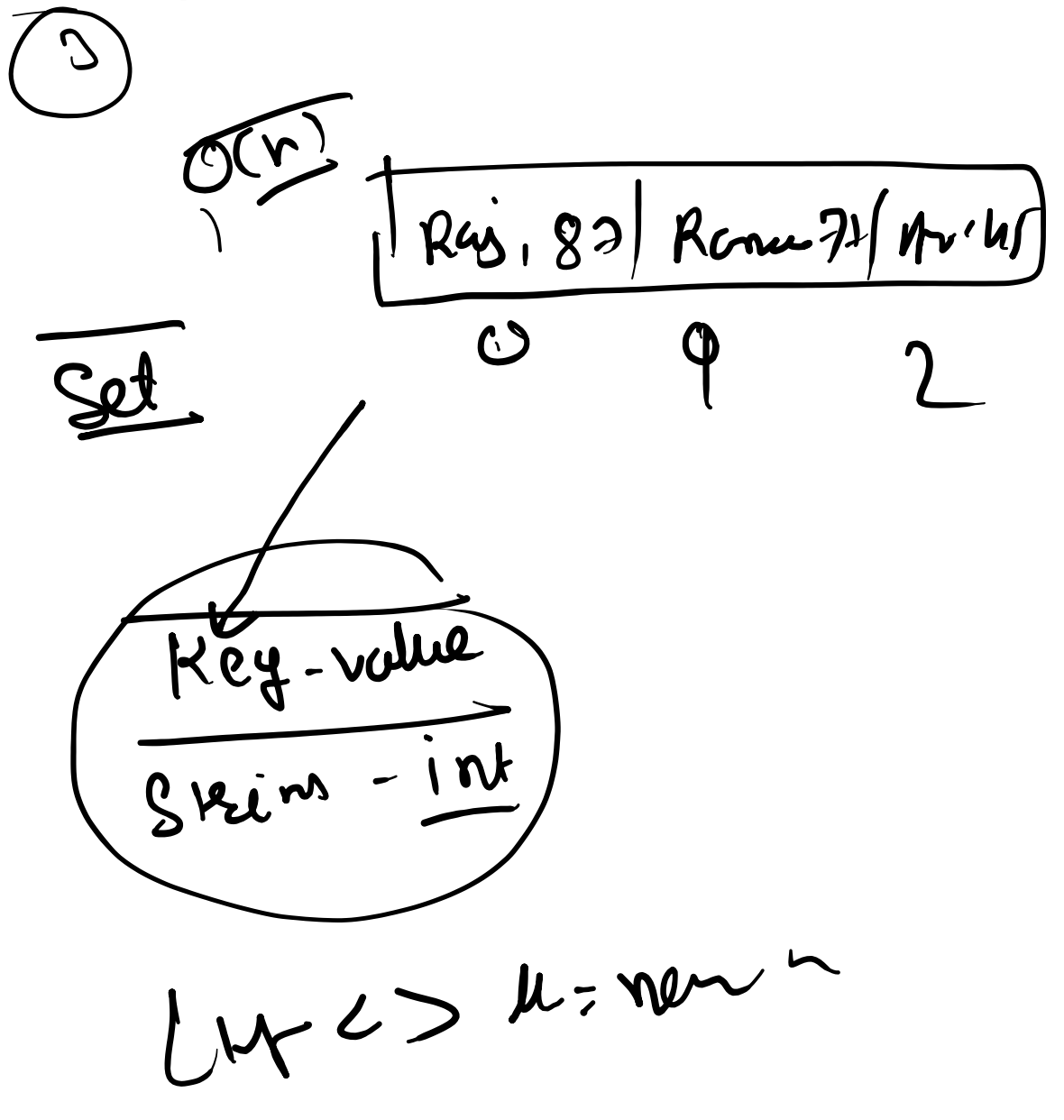
```
6
7 9
0 10
4 5
8 9
4 10
5 7
```


4 5
5 7
8 9
7 9
0 10
4 10

```
int activitie = 1;
    int end = arr[0].et;
    for (int i = 1; i < arr.length; i++) {
        if (end <= arr[i].st) {
            end = arr[i].et;
            activitie++;
        }
    }
    System.out.println(activitie);
```

3


interface
Map
need
O(1)
HashMap
TreeMap
LinkedHashMap


O(n)
Raj, 82
Raman 71
Ankit
0
1
2
Set
Key - value
String - int

LL<> ll = new

O(1) HashSet ← Set

log(n) TreeSet

O(1) LinkedHashSet

<In>

int[] arr = new int[5]

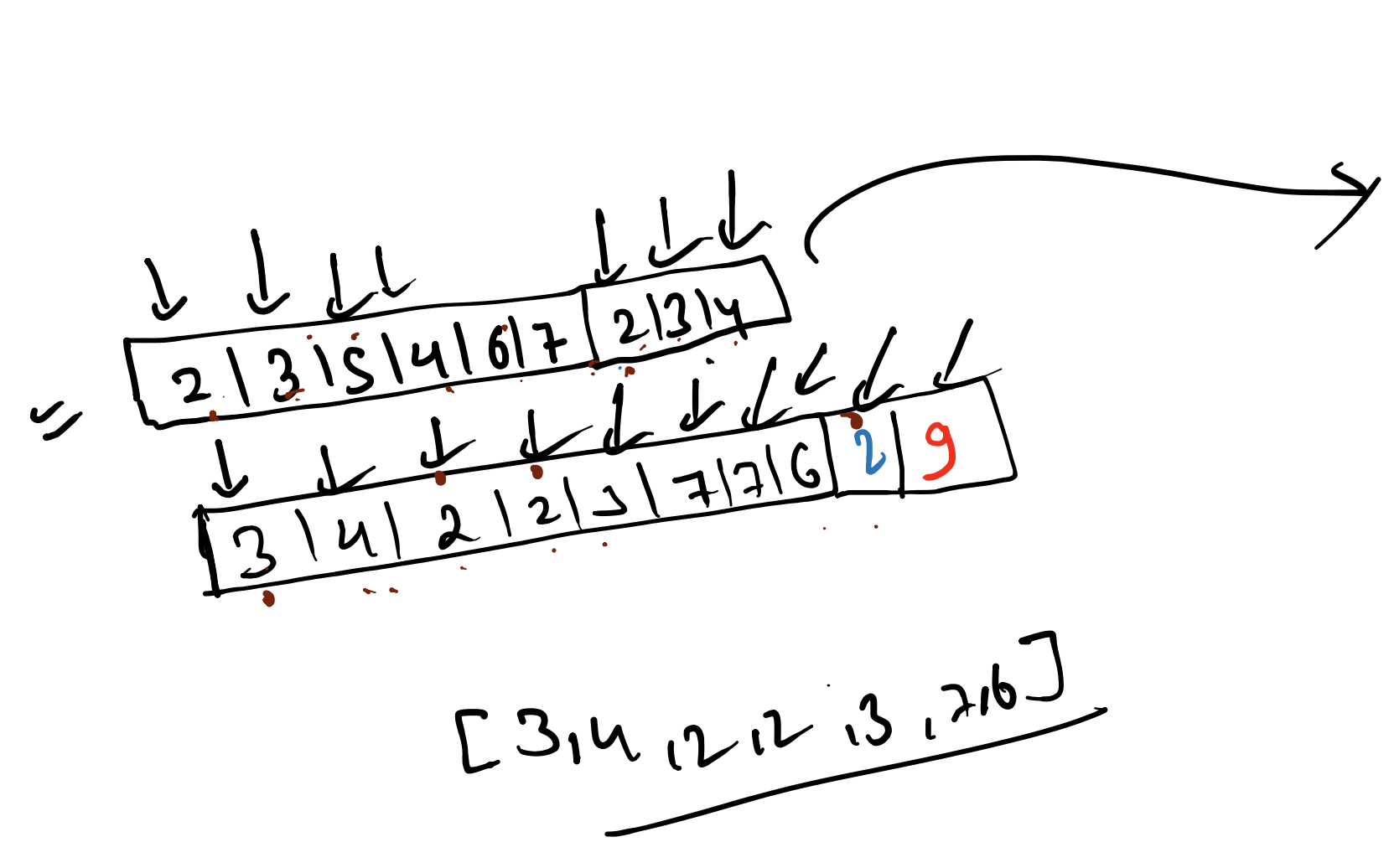
ArrayList<Integer> ll = new ArrayList<>()

for(int x: ll) {

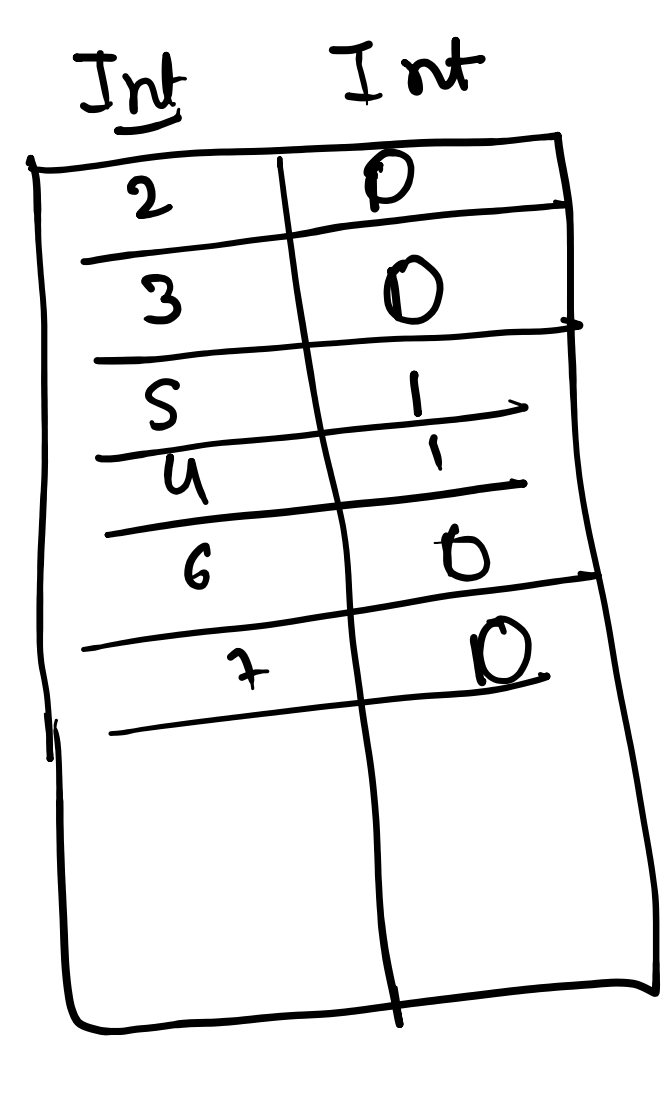
}

for(int x: arr) {

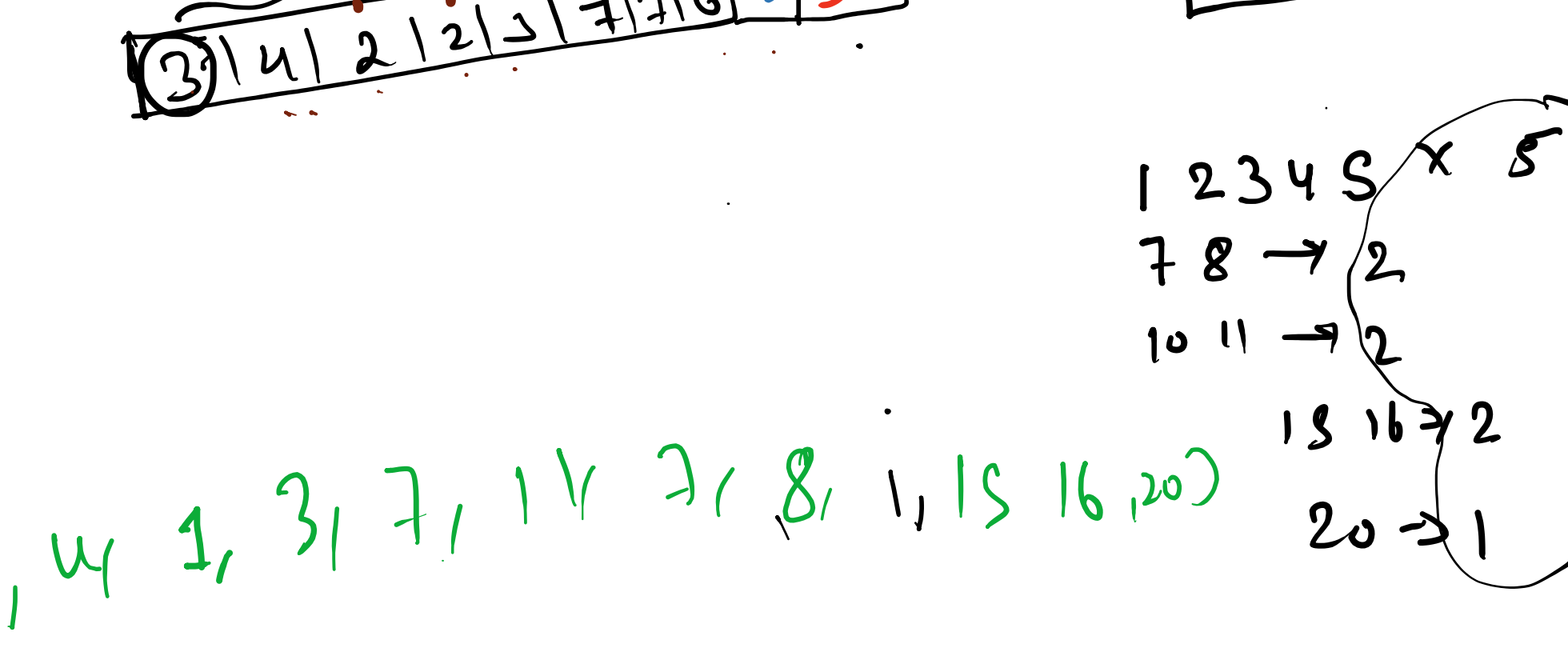
}

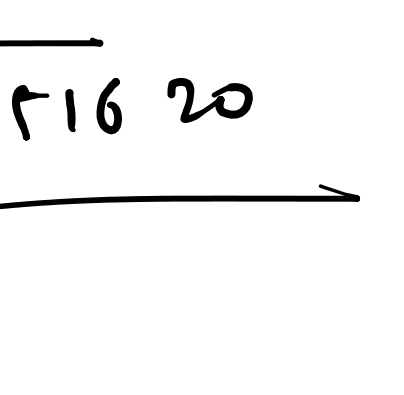

2 3 5 4 6 7 2 3 4
3 4 2 2 3 7 7 6 2 9

[3, 4, 2, 2, 3, 7, 6]


Int
Int
2
0
3
0
5
1
4
1
6
0
7
0

```
public static int[] Intersection_of_Two_Arrays(int[] arr1, int[] arr2) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < arr1.length; i++) {
            if (map.containsKey(arr1[i])) {
                map.put(arr1[i], map.get(arr1[i]) + 1);
            } else {
                map.put(arr1[i], 1);
            }
        }
    }
```


2 3 5 4 6 7 2 3 4
3 4 2 2 3 7 7 6 2 9
2
2
3
2
5
1
4
2
6
1
7
1


1 2 3 4 5 → 5
7 8 → 2
10 11 → 2
15 16 → 2
20 → 1
5

[2, 2, 5, 10, 4, 1, 3, 7, 11, 7, 8, 1, 15, 16, 20]

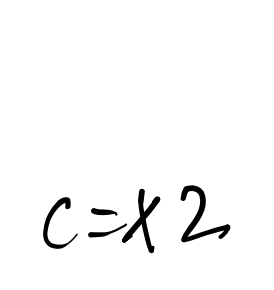
1 1 2 2 3 4 5 7 8 10 11 15 16 20 → 1 2 3 4 5 7 8 15 16 20

ans = ~~1~~ 5

[2, 2, 5, 10, 4, 1, 3, 7, 11, ~~7~~, ~~8~~, ~~1~~, ~~15~~ 16, 20]

c = ~~1~~ 2 1 2 1

c = 1 2 3 4 5


1 2 3 4
5 7 8
10 11
15 16 20

```
public static int Longest_Consecutive(int[] arr) {
        HashSet<Integer> set = new HashSet<>();
        for (int i = 0; i < arr.length; i++) {
            set.add(arr[i]);
        }
        int ans = 0;
        for (int i = 0; i < arr.length; i++) {
            int x = arr[i];
            if (!set.contains(x - 1)) {
                int c = 0;
                while (set.contains(x)) {
                    set.remove(x);
                    c++;
                    x++;
                }
                ans = Math.max(ans, c);
            }
        }
        return ans;
    }
```

c = ~~1~~ 2   ans = ~~2~~ 5

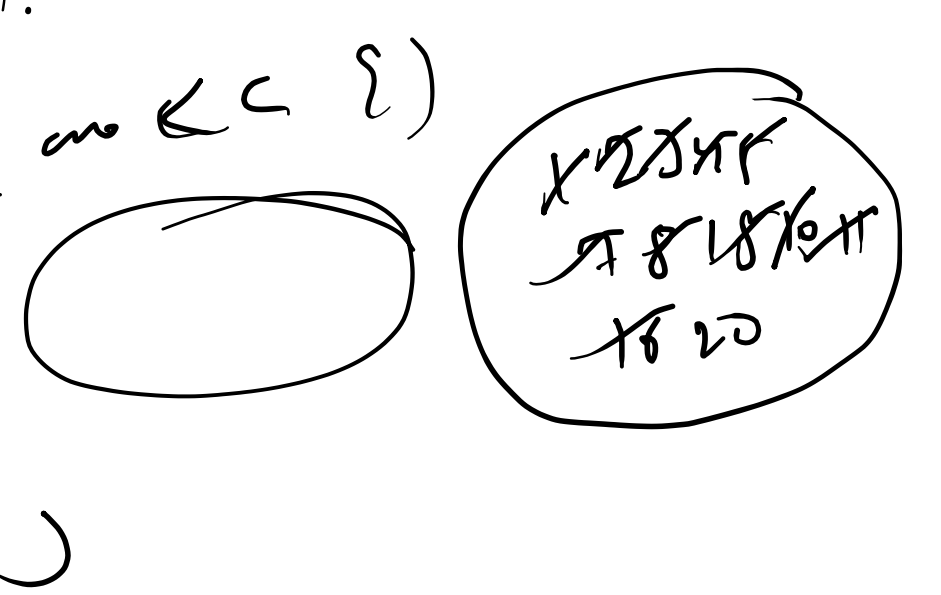
c = ~~0~~ 1 2   x = ~~7~~ 8 9

c = ~~0~~ 1 2   x = ~~1~~ 2 3 4 5 6

x = ~~1~~ 16

[2, 2, 5, 10, 4, 1, 3, 7, 11, 7, 8, 1, 15, 16, 20]

if(ans < c){


1 2 3 4 5
7 8 10 11
16 20